

GymBuddy

Technical Documentation Guide

Aidan Lilly, Hayden Mann, Alex Hanson, Louis Crabtree

Overview.....	3
System Requirements.....	3
Installation.....	3
Environment Setup.....	3
Network Configuration.....	4
Running the Application.....	4
Database Behavior.....	5
Authentication System.....	5
QR Code System.....	5
Machine Queue System.....	6
Admin / Maintenance Functions.....	6
Testing System.....	6
Deployment Considerations.....	7
Troubleshooting.....	7
Developer Notes.....	8

Overview

GymBuddy is a full-stack gym management system that provides:

Real-time machine queue tracking

Secure QR-based check-in system

JWT-based authentication with protected routes

Workout plan management

Fully tested backend using Jest + Supertest

This guide allows a developer to install, configure, run, and maintain the application.

System Requirements

Node.js (v18+ recommended)

npm

PostgreSQL

Git

Installation

Clone the repository and install dependencies:

```
git clone <repo-url>
cd cs-capstone-project-group-1
npm install
npm run install-all
```

Environment Setup

Create a .env file inside the Backend folder:

Backend/.env

Use the following configuration:

```
PORT=5001
JWT_SECRET=your-secret-here
```

```
PGHOST=localhost
PGPORT=5000
PGUSER=postgres
PGPASSWORD=your_password
PGDATABASE=gymbuddy
DB_SSL=false
```

- Important Notes

Backend runs on port 5001

PostgreSQL runs on port 5000 (not default 5432)

The backend will NOT start without a valid .env file

Network Configuration

Edit the file:

```
GymBuddy/src/config/network.config.ts
```

Set your computer's IP address:

```
COMPUTER_IP = "your-local-ip"
```

To find your IP:

Mac: `ifconfig | grep inet | grep -v 127.0.0.1`

Windows: `ipconfig`

Running the Application

Start backend in /backend

```
npm run dev
```

Start frontend in /gymbuddy

```
npm start
```

This will:

Start the Express backend

Start the Expo frontend

Database Behavior

The database is automatically handled by the backend.

On startup, the system:

- Creates the database if it does not exist
- Creates all required tables
- Seeds initial machine data

No manual SQL execution is required.

Tables Created

- users
- machines
- machine_queue
- machine_usage
- workout_plans
- workout_plan_items

Authentication System

GymBuddy uses JWT (JSON Web Tokens) for authentication.

Flow:

1. User registers or logs in
2. Server generates a JWT
3. Token is sent with API requests
4. Middleware validates token
5. Access is granted or denied

Example Protected Route:

/api/protected

QR Code System

Features:

- Requires authentication
- Generates QR codes

- Includes expiration logic
- Prevents unauthorized reuse

Workflow:

1. User logs in
2. User generates QR code
3. QR expires after a short duration
4. Used for gym check-in

Machine Queue System

Features:

- Tracks current machine user
- Allows users to join queue
- Maintains queue order
- Updates dynamically

Workflow:

1. User selects machine
2. User joins queue
3. Queue updates in real time
4. Next user is promoted when machine is free

Admin / Maintenance Functions

There is no dedicated admin UI.

Administrative actions are performed through:

- Database queries

- Server-side logic

Example Queries:

```
SELECT FROM users;
```

```
SELECT FROM machine_queue;
```

```
DELETE FROM machine_queue WHERE machine_id = 1;
```

Testing System

Run all tests:

```
npm test
```

Run tests with coverage:
npm test -- --coverage

Test Results

13 test suites
16 tests
All tests passing

What is Tested

Authentication
JWT validation
Protected routes
QR generation
Input validation
Workout plans

Deployment Considerations

- Security
 - Use strong JWT_SECRET
 - Enable HTTPS in production
 - Validate all user inputs
- Environment
 - Do NOT commit .env
 - Use secure environment variables in production
- Deployment Options
 - AWS / EC2
 - DigitalOcean
 - Render / Railway

Troubleshooting

Server will not start

npm install
npm run install-all

Database connection failed

Ensure PostgreSQL is running

Verify port = 5000
Verify credentials in .env

JWT errors

Ensure JWT_SECRET is set
Ensure token is included in request headers

QR not generating

Ensure user is authenticated
Ensure token is valid

Port already in use

Change .env:
PORT=3000

Developer Notes

Backend location:

GymBuddy/src/actions

Key files:

server.ts (main server)
routes/ (API endpoints)
controllers/ (business logic)
middleware/ (auth + security)
GymBuddy/tests